REYNAUD Nathan
SUNDMAN--MATHON Thomas
DUMARCHÉ Dorian
TESTELIN Maxime
Marzaq Hamza
Mohamed Nezhari

Compte Rendu/ Journal de bord
Éditeur de liens: Fusion

Introduction

Ce projet a pour objectif de développer un éditeur de liens capable de lire et ensuite fusionner des fichiers au format ELF (Executable and Linkable Format) 32 bits pour l'architecture ARM en big endian. Il se décompose en deux phases principales.

La première phase concerne la lecture e et l'affichage des différentes composantes d'un fichier ELF : en-tête, table des sections, contenu de section, table des symboles et tables de réimplantation.

La seconde phase (étapes 6 à 9) implémente la fusion de deux fichiers ELF en un seul. Cette fusion inclut la concaténation des sections des 2 fichiers, la renumérotation et correction des symboles, et l'ajustement des tables de réimplantation. Le résultat final est un fichier ‘fusion.o’, comparable à celui produit par ‘arm-none-eabi-ld’.

Ce compte-rendu présente notre implémentation, détaille la structure du code, la liste des fonctionnalités réalisées et manquantes, identifie les bogues connus, et décrit les tests effectués pour valider notre solution.

Phase 1

Étape 1 : Affichage du header d’un .o .(Nathan)

Pour effectuer cette première étape, j'ai utilisé la commande : gcc -c sur un fichier .c pour obtenir un fichier objet, que j'ai ensuite donné en argument à la commande readelf -h afin d'obtenir le résultat de l'affichage voulu.
J'ai ensuite parcouru le fichier elf.h dans /usr/include/ ainsi que la documentation du format ELF afin de savoir où récupérer les informations pour construire notre propre affichage de l'en-tête.
J'ai donc ouvert le fichier .o à l'aide de fopen, puis j'ai créé une variable header, de type Elf32_Ehdr qui est le type utilisé pour stocker le header, j'ai donc inclus elf.h dans le programme.

j'ai utilisé la fonction fread pour lire l'en-tête du fichier .o puis stocké le résultat de la lecture dans header pour pouvoir parcourir les informations du header.
J'ai utilisé les reverse_2(x) et reverse_4(x) contenu dans le fichier util.h afin de pouvoir respecter l'affichage que la machine hôte du programme soit en little ou en big endian.

Afin de vérifier si mon code marchait j'ai transformé le fichier .c en .o de plusieurs manières différentes. Avec gcc de manière classique, en forçant l'utilisation de 32bits à la place de 64 avec l'option *-m32* (en ayant au préalable installé le package gcc-multilib) puis en utilisant arm-none-eabi-gcc avec les options *-marm -mbig-endian* pour respecter les specs du projet.

Après l'audit, nous avons séparé la lecture et l'affichage et utilisé notre struct elf_contexte_t à la place d'utiliser directement Elf32_Ehdr.

```
noma@Ubuntu24:~/Documents/CoursC/fusion$ readelf -h  utilbe32.o
En-tête ELF:
  Magique:   7f 45 4c 46 01 02 01 00 00 00 00 00 00 00 00 00
  Classe:                            ELF32
  Données:                           complément à 2, système à octets de poids fort d'abord (big endian)
  Version:                           1 (actuelle)
  OS/ABI:                            UNIX - System V
  Version ABI:                       0
  Type:                              REL (Fichier de réadressage)
  Machine:                           ARM
  Version:                           0x1
  Adresse du point d'entrée:               0x0
  Début des en-têtes de programme :          0 (octets dans le fichier)
  Début des en-têtes de section :          512 (octets dans le fichier)
  Fanions:                           0x5000000, Version5 EABI
  Taille de cet en-tête:             52 (octets)
  Taille de l'en-tête du programme:  0 (octets)
  Nombre d'en-tête du programme:     0
  Taille des en-têtes de section:    40 (octets)
  Nombre d'en-têtes de section:      10
  Table d'index des chaînes d'en-tête de section: 9
noma@Ubuntu24:~/Documents/CoursC/fusion$ ./header ./utilbe32.o
En-tête ELF:
Magique:   7f 45 4c 46 01 02 01 00 00 00 00 00 00 00 00 00
Classe:   ELF32
Données:   Big Endian
Version:   1
OS/ABI:   0
Version ABI:   0
Type:   0x0001 REL (Fichier de réadressage)
Machine:   0x0028 ARM
Version:   0x00000001
Adresse du point d'entrée:   0x00000000
Début des en-têtes de programme:   0 (octets dans le fichier)
Début des en-têtes de section:   512 (octets dans le fichier)
Flags:   0x05000000
Taille de cet en-tête: 52 (octets)
Taille de l'en-tête du programme:   0 (octets)
Nombre d'en-tête du programme:   0
Taille des en-têtes de section:   40 (octets)
Nombre d'en-tête de section:   10
Table d'index des chaînes d'en-tête de section:   9
```

Doc utilisés:

[arm-none-eabi-readelf(1)]

[Documentation format ELF]

[elf.h]

[Force gcc to compile 32 bit programs on 64 bit platform - Stack Overflow]

[Appendix B: Assigned OSABI Values — ELF Object File Format 4.3 DRAFT documentation]

[EDL Fusion]

Etape 2: Affichage de la table des sections. (Nathan)

Pour réaliser l'étape 2 j'ai récupéré les informations nécessaires à l'affichage de la table des sections depuis le header (Etape 1), précisément le nombre de sections (e_shnum) et l'offset de la table des sections (e_shoff). J'ai ensuite lu la table des sections (Elf_Shdr) afin de récupérer chaque information de chaque section. J'ai aussi récupéré le nom des sections depuis .shstrtab, j'ai réalisé un affichage similaire à la fonction readelf.

Etape 3 : Affichage contenu d'une section (Thomas)

Pour réaliser l'étape 3 on utilise le elf_contexte_t déjà en partie remplis dans les étapes précédente puis on lit le nom ou l'indice de section donnée en paramètre et on vérifie avec strtol si c'est un nombre ou un nom qui à été donner et si c'est un nom on cherche l'indice de la section ayant ce nom avec trouver_section() puis on verifie que l'indice est bien valide et si oui on affiche le contenu en hexadécimal de la section.

Etape 4 : Affichage de la table des symboles (Mohamed, Dorian)

Pour réaliser l'étape 4, on commence par charger la table des symboles dans la struct elf_contexte_t en s'aidant du travaille fait dans l'étape 3 puis on l'afficher avec toute ses informations en faisant attention pour le nom car quand le symbole est un nom de section il faut le chercher dans le nom_sections et quand ce n'est pas une section il faut chercher le nom dans nom_symboles.

Etape 5 : Affichage de la table des reimplantations (Maxime)

Pour réaliser l'étape 5, j'ai travaillé dans la continuité de l'étape 4; le processus était similaire, lire des informations, les charger et les traiter; il fallait par contre que la table des symboles soit chargé pour les valeurs et noms des symboles reimplantés. J'ai séparé en REL et RELA.

Phase 2

Étape 6 : Fusion des tables de section (Nathan, Thomas)

L'objectif est de fusionner les sections de type PROGBITS de 2 fichiers ELF.

Pour chaque section :

- Si elle n'est présente que dans un seul fichier, elle est conservé telle quelle
- Si elle est présente dans 2 fichiers alors on fusionne en concaténant la section du fichier 2 à la suite de celle du fichier 1.

J'utilise une structure intermédiaire pour stocker les informations nécessaire (section_fusion_t) qui permet de stocker :

- Le nom de la section
- Son index dans le fichier 1 et dans le fichier 2
- Sa taille dans chaque fichier
- L'offset de concaténation

Lors de la construction de la table de sections de sortie on modifie sh_size pour mettre à jour la taille des sections fusionner.

f

Etape 7: Fusion des tables de symboles (Maxime)

Pour cette section, j'ai utilisé une structure pour stocker un tableau de symboles, le nombre de symboles et la table de noms associés. J'ai procédé en 3 étapes :

- Séparer les symboles locaux des symboles globaux
- Fusionner les tables globales, en suivant les règles de fusion associées
- Ajouter les tables locales 1 puis 2, suivi de la table pré fusionée globale.

Le résultat est une structure, qui pourrait ensuite être écrite dans le fichier.

Etape 8 :Fusion des tables de réimplantations(Mohamed, Hamza)

Principe : 1. Les tables de même nom (ex: .rel.text) sont fusionnées par concaténation 2. Pour chaque entrée de réimplantation, trois corrections sont nécessaires : - Correction de l'offset : ajouter l'offset de concaténation de la section - Correction de l'indice du symbole : mapper vers la table de symboles fusionnée - Correction de l'addend (pour RELA avec symboles SECTION) Algorithme : - Créer un mapping entre anciens et nouveaux indices de symboles - Pour chaque table de réimplantation : * Si elle existe dans les deux fichiers : fusionner les entrées * Sinon : copier la table unique - Corriger chaque entrée selon son origine (ctx1 ou ctx2) Difficultés rencontrées : - Gestion de la mémoire : éviter la destruction des contextes originaux - Sections debug : filtrage nécessaire dans l'étape 6 (fusion_table.c) - Mapping des symboles : respecter l'ordre LOCAL puis GLOBAL Résultat : Tables de réimplantation fusionnées avec offsets et symboles corrigés, prêtes pour l'étape 9 (génération du fichier ELF final). Test : Comparaison avec arm-none-eabi-ld -r confirme la correction.

Etape 9: Production d’un fichier fusion (Dorian)

(une partie de l’étape 9 avait déjà été réalisés dans l’étape 6)

Cette étape à directement été réalisé dans le elf_reader.c et fichier de sortie = fusion.o :

- On écrit d’abord un header provisoir qui va être modifié plus tard
- On créer la table des sections header appeler full_sections
- on ajoute dans la table d'abord les sections fusionnés dans l’étape 6 puis celle de l’étape 7 et enfin la table des sections elle même en faisant attention de bien avoir les bon offsets
- on modifie le headers pour qu’il donne la bonne position de la table des sections

Problème :

Dans l’étape 7 quand le symbole est une section le numéro n’a pas été changé donc certains ont les mauvais noms à l’affichage.

Il y a eu des erreurs dans la partie 8 donc il n’était pas possible de l’intégrer dans l’écriture du fichier fusionné.